PF00 -Actividad final Tipo R – DTPR

Integrantes:

- Carlos Andrés Rodríguez Pelayo
- Nestor Andrés Rivera Quintana
- Gerardo Josué Morales Sánchez
- Jared Vera Jaime

INTRODUCCION

Instrucciones tipo r

Las instrucciones de tipo R son aquellas en donde todos los operandos se encuentran en el banco de registros. En la arquitectura MIPS, que es la base de para este tipo de instrucciones, su arquitectura se divide de la siguiente manera:

Una entrada de 32 bits divididos en distintas secciones:

- Operation code. En las instrucciones de tipo r siempre es 000000. Sus 6 bits son enviados como la única entrada a la Unidad de control.
- rs, rt y rd. En donde rs y rt son los registros fuentes, rd es el registro destino. En donde cada uno de estos 3 registros son de 5 bits.
- Shift Amount (Shamt). Dependiendo del tipo de operación a realizar, este dato se llena con el número de desplazamientos por hacer. En este caso puntual, ninguna de las 5 operaciones utilizadas (SUB, ADD, AND, OR y SLT) necesitan la utilización de shamt, por lo que se dejan sus 5 bits en cero: 00000.
- Function. Es el código especial de cada una de las operaciones aritméticas. Se conforma de 6 bits. Este dato es una de las dos entradas que tendrá más delante la ALU control. Indicando en todo momento mediante sus bits, la operación a realizar.

Para ejemplificar una de las operaciones a realizar, el operador SLT (set less than), cuyo código binario de 6 bits en el function es: 6’b101010, necesita dos

registros para comparar. En este caso se utiliza rs < rt, al compararlos, si es verdadero, rd se carga con un 1 lógico, en el caso contrario se carga con 0.

Operación Ternaria.

La operación ternaria usa tres elementos, si se pudiese comparar con alguna estructura, sería la if-else. Pues dentro de la implementación en código, por ejemplo, dicha estructura evalúa una condición, dependiendo de esto, el programador asigna una acción si la sentencia es verdadera, si es falsa, se utilizaría else para añadir otro resultado. En el operador ternario, su sintaxis se compone de tres operandos: una condición lógica seguida de un signo de interrogación (?), el valor resultante si la condición es verdadera, un signo de dos puntos (:) y el valor si es falsa. Esto sirve mucho para ahorrar espacio y generar un bloque optimizado de código que evalúa una condición y se le asigna una, además de que lo hace inmediatamente, contrario al caso de if-else que su uso no retorna automáticamente una respuesta.

Proceso de compilación

El proceso de compilación es la transformación de un lenguaje de alto nivel a un lenguaje que pueda ser procesada por el hardware. El compilador verifica que la sintaxis, variables y tipos de variables entre otras cosas sean correctas y que no pueda haber errores. El proceso de compilación pasa por distintas fases, y en última instancia transforma la información a lenguaje ensamblador y posteriormente a binario para ser procesado y ejecutado por el hardware.

En el contexto de esta actividad, el proceso de compilación puede ser ejemplificado en como traduce el operador ternario (condición) ? true : false. A una secuencia de instrucciones que pueda ser ejecutada por el datapath.

OBJETIVO

El objetivo comprende distintas capas. Creemos que es fundamental comprender como funcionan los lenguajes de descripción de hardware como Verilog, para comprender al mismo tiempo, cómo funciona el hardware a nivel ingeniería. Comprendiendo como es procesado cada bit utilizado, y por cuales procesos pasa. Es importante reconocer la utilidad de las variables y sintaxis del lenguaje para construir proyectos, comenzando con cada uno de los módulos, y

posteriormente, relacionándolos, conectándolos de una u otra manera, ya sea instanciado o conectado con wires. Lo anterior es la última parte de la capa a la hora de trabajar con lenguajes de descripción de hardware, pues se verifica la teoría en la práctica, utilizando simuladores como ModelSim para verificar la correcta funcionalidad de los módulos trabajando juntos.

DESARROLLO

Comenzamos con el desarrollo de cada uno de los siguientes módulos independientes:

- ALU
- ALU control
- Banco de registros
- Memoria
- Multiplexor 2 a 1
- Unidad de control

Ya contábamos con algunos de los módulos, hechos en actividades y clases anteriores, como el caso de la ALU. Algunos otros módulos fueron facilitados por el profesor como el Banco de registros, realizado en clases anteriores para introducir el tema de las instrucciones de tipo r.

Gracias a la experiencia generada a lo largo del semestre, fuimos capaces de generar el resto de módulos, como Unidad de control, mux, memoria y ALU control. Es importante destacar que para la realización de cada módulo, es importante conocer el problema a resolver, pero también, la sintaxis y aplicación del lenguaje de descripción de hardware verilog. Gracias a que conocemos la estructura de declaración de entradas y salidas, variables, y bloques secuenciales, fuimos capaces de implementar cada código según su necesidad de implementación dentro del módulo general DPTR, ayudándonos principalmente del contenido visto en clases, explicado por el profesor, pero como ayuda gráfica, nos guiamos al 100% por la fotografía contenida en el documento adjunto proveído por el profesor.

Para instanciar cada uno de los módulos en uno que los unificara para completar el proyecto, utilizamos un nuevo módulo llamado DPTR. Utilizamos los módulos existentes como ya fue mencionado, los instanciamos y además declaramos wires para conectar entradas y salidas, basándonos enteramente en la fotografía que ha sido referenciada varias veces. Si se observa la similitud entre el nombre de nuestros wires en el DPTR, con el nombre de los wires en la imagen referenciada, podemos encontrar coincidencia en el nombre de cada cable nombrándolos C_n, en donde n es el número de cada cable utilizado para conectar entradas y salidas, comenzando por 1.

Algunos otros cables fueron mencionados por el profesor durante las clases, pero no se vieron reflejados en el diagrama, por lo que lo tuvimos que añadir y usar nuestros conocimientos para hacerlo formar parte del sistema en general. En este caso en particular hacemos referencia al wire llamado C_MemToRead. Este wire es utilizado como salida de la unidad de control, con 1 bit de tamaño, y es entrada, junto con otros inputs provenientes de distintos módulos, al módulo de Memoria.

Para finalizar con la implementación practica del proyecto, realizamos el TestBench, con las instrucciones especificadas en el documento. Estos fueron los pasos realizados:

1. Realizamos la tabla de instrucciones en formato ensamblador y su equivalente en código máquina.

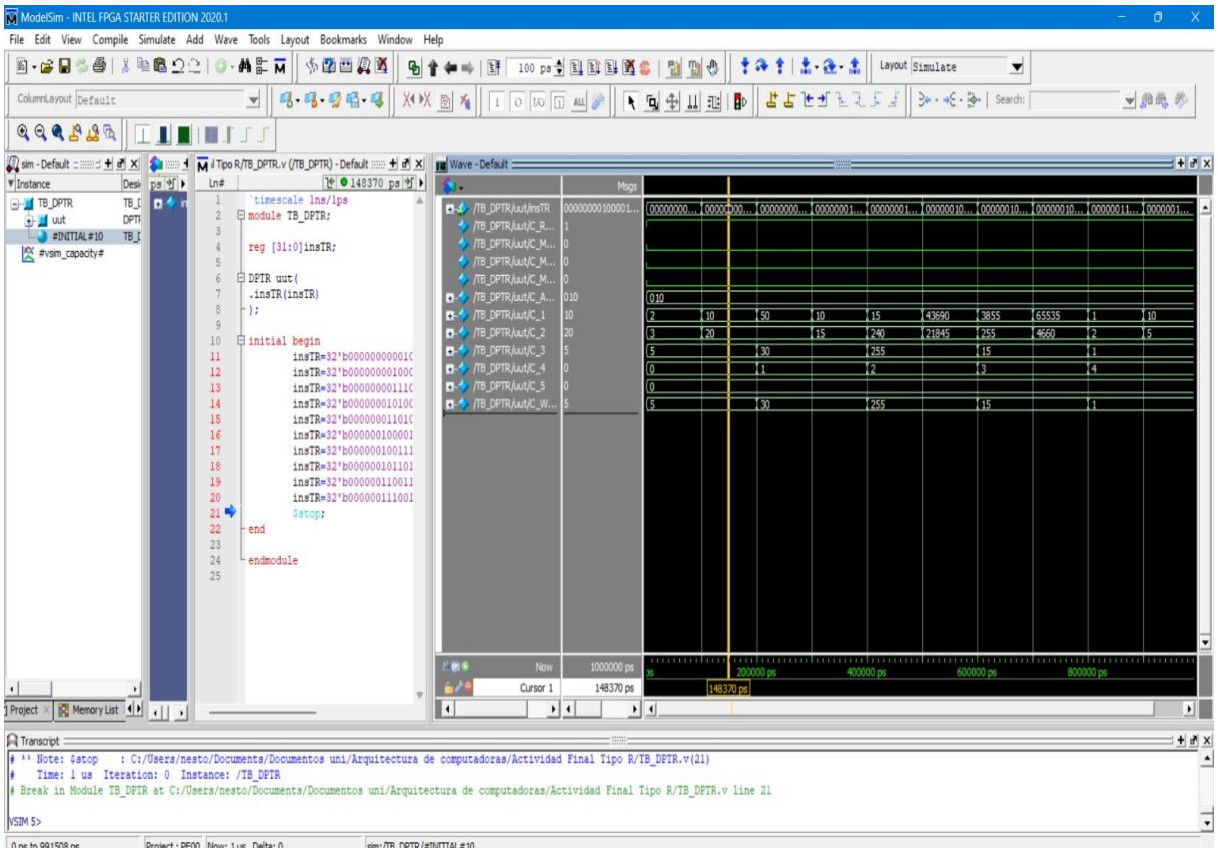
Instrucciones en Ensamblador

#	Instrucción Ensamblador	Operación Lógica / Matemática	OpCode (6 bits)	RS (5 bits)	RT (5 bits)	RD (5 bits)	Shamt (5 bits)	Funct (6 bits)
1	ADD R3, R1, R2	R3 = R1 + R2	0	00001 (R1)	00010 (R2)	00011 (R3)	0	100000
2	ADD R6, R4, R5	R6 = R4 + R5	0	00100 (R4)	00101 (R5)	00110 (R6)	0	100000
3	SUB R9, R7, R8	R9 = R7 - R8	0	00111 (R7)	01000 (R8)	01001 (R9)	0	100010
4	SUB R12, R10, R11	R12 = R10 - R11	0	01010 (R10)	01011 (R11)	01100 (R12)	0	100010
5	OR R15, R13, R14	R15 = R13 | R14	0	01101 (R13)	01110 (R14)	01111 (R15)	0	100101
6	OR R18, R16, R17	R18 = R16 | R17	0	10000 (R16)	10001 (R17)	10010 (R18)	0	100101
7	AND R21, R19, R20	R21 = R19 & R20	0	10011 (R19)	10100 (R20)	10101 (R21)	0	100100
8	AND R24, R22, R23	R24 = R22 & R23	0	10110 (R22)	10111 (R23)	11000 (R24)	0	100100
9	SLT R27, R25, R26	R27 = (R25 < R26) ? 1 : 0	0	11001 (R25)	11010 (R26)	11011 (R27)	0	101010

Instrucciones en Código Máquina

#	Instrucción Ensamblador	Código Binario de 32 bits	Código para TestBench
1	ADD R3, R1, R2	000000_00001_00010_00011_00000_100000	32'b00000000000100010000110000010000100000
2	ADD R6, R4, R5	000000_00100_00101_00110_00000_100000	32'b00000000010000101001100000010000100000
3	SUB R9, R7, R8	000000_00111_01000_01001_00000_100010	32'b00000000011101000010010000010000100010
4	SUB R12, R10, R11	000000_01010_01011_01100_00000_100010	32'b00000000101001011011000000010000100010
5	OR R15, R13, R14	000000_01101_01110_01111_00000_100101	32'b00000000110101110011110000010010100101
6	OR R18, R16, R17	000000_10000_10001_10010_00000_100101	32'b00000001000010001100100000010010100101
7	AND R21, R19, R20	000000_10011_10100_10101_00000_100100	32'b00000001011101001010100000010010100100
8	AND R24, R22, R23	000000_10110_10111_11000_00000_100100	32'b00000010110101111100000000100100100100
9	SLT R27, R25, R26	000000_11001_11010_11011_00000_101010	32'b00000001100111010110110000010101010
		000000_11100_11101	

2. En la columna “Código para TestBench” se aprecia el código que fue ingresado como entrada al TestBench, y nos arrojó los siguientes resultados.



CONCLUSION

Creemos que después de haber realizado esta actividad se resolvieron en nosotros algunas dudas como la importancia del acomodo de bits, pero sobre todo, como cada uno de estos bits es utilizado según su número, pues, por ejemplo en el acomodo de cada uno de los 32 bits de entrada [31:0]. Se puede observar posteriormente como en la separación de estos bits en grupos de 6 y 5 bits, se va restando y separando, hasta llegar al bit 0. Esto es más demostrable y entendible al instanciar los módulos que contienen esto.

Creemos que en esta ocasión nos fue más fácil de comprender como realizar un testbench, sumado a la experiencia de la última vez que realizamos uno, inclusive sabiendo que en este la dificultad técnica podría ser un poco más alta

debido al requerimiento de ingresar como entrada al testbench los datos en código máquina.

Dentro de la parte de investigación, nos parece curioso cómo se requiere la investigación del operador ternario y del proceso de compilación, que si bien, forman parte del marco teórico general por su importancia en la realización de tareas a la hora de utilizar una computadora, no lo habíamos visto requerido anteriormente en clases para la comprensión del tema aquí presentado, sin embargo, fueron útiles para comprender como funciona la compilación respectivamente, y como se ejemplifica mejor el uso de un SLT o una estructura if-else, todas estas estructuras indispensables a la hora de realizar un proyecto.

BIBLIOGRAFIA

Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2.ª ed.). Prentice Hall.

Patterson, D. A., & Hennessy, J. L. (2017). *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann.

Baeldung. (2024, 15 de marzo). *How Compilers Work*. Baeldung on Computer Science. https://www.baeldung.com/cs/how-compilers-work